Deliverable #2

SE 3A04: Software Design II – Large System Design

Tutorial Number: T02
Group Number: G1
Group Members:

- Prutha Patel
- Kaiden Jessani
- Irfan Shahid
- Christopher Ludwig
- Mohammad Bilal

1 Introduction

1.1 Purpose

The purpose of this document is to describe the high-level architecture of the Smart City Environmental Monitoring & Alert System (SCEMAS). It discusses important system design considerations and provides an overview of the key subsystems that support the platform.

This document is intended for internal stakeholders at SCEMAS, including project managers, developers, domain experts, and investors. Familiarity with SCMEAS deliverable 1 is recommended, as it provides background information relevant to the architectural discussion.

1.2 System Description

The Smart City Environmental Monitoring & Alert System (SCEMAS) is a cloud-based, IoT-integrated platform designed to collect, process, and manage environmental telemetry data from sensors placed throughout a city. The system provides real-time data visualization to the public through an online dashboard and will allow users to be alerted about severe environmental data. SCEMAS tracks air quality, temperature, humidity, and noise levels, transforming this data into actionable information for city operators and increasing citizen awareness.

SCEMAS follows a PAC architecture style, which allows the system to separate presentation concerns from core business logic while supporting multiple concurrent data streams and real-time responsiveness. The system is further broken down into the subsystems, sensor service, alerting service, account service, and operational module, which follow a pipe and filter, blackboard, repository, and repository architecture style, respectively. This design ensures high cohesion in the subsystems and low coupling between them, allowing for maintainability and scalability.

1.3 Overview

Section 2 includes an analysis class diagram for the Smart City Environmental Monitoring & Alert System (SCEMAS) detailing the overall structure of the program. Section 3 provides a thorough description of the architectural design of the system, presenting various subsystems, their purpose, and architectural style, as well as reasoning for these choices. This section also includes a structural architecture diagram for the system. Finally, section 4 consists of class responsibility collaboration (CRC) cards to provide further details of each class and their interactions.

2 Analysis Class Diagram

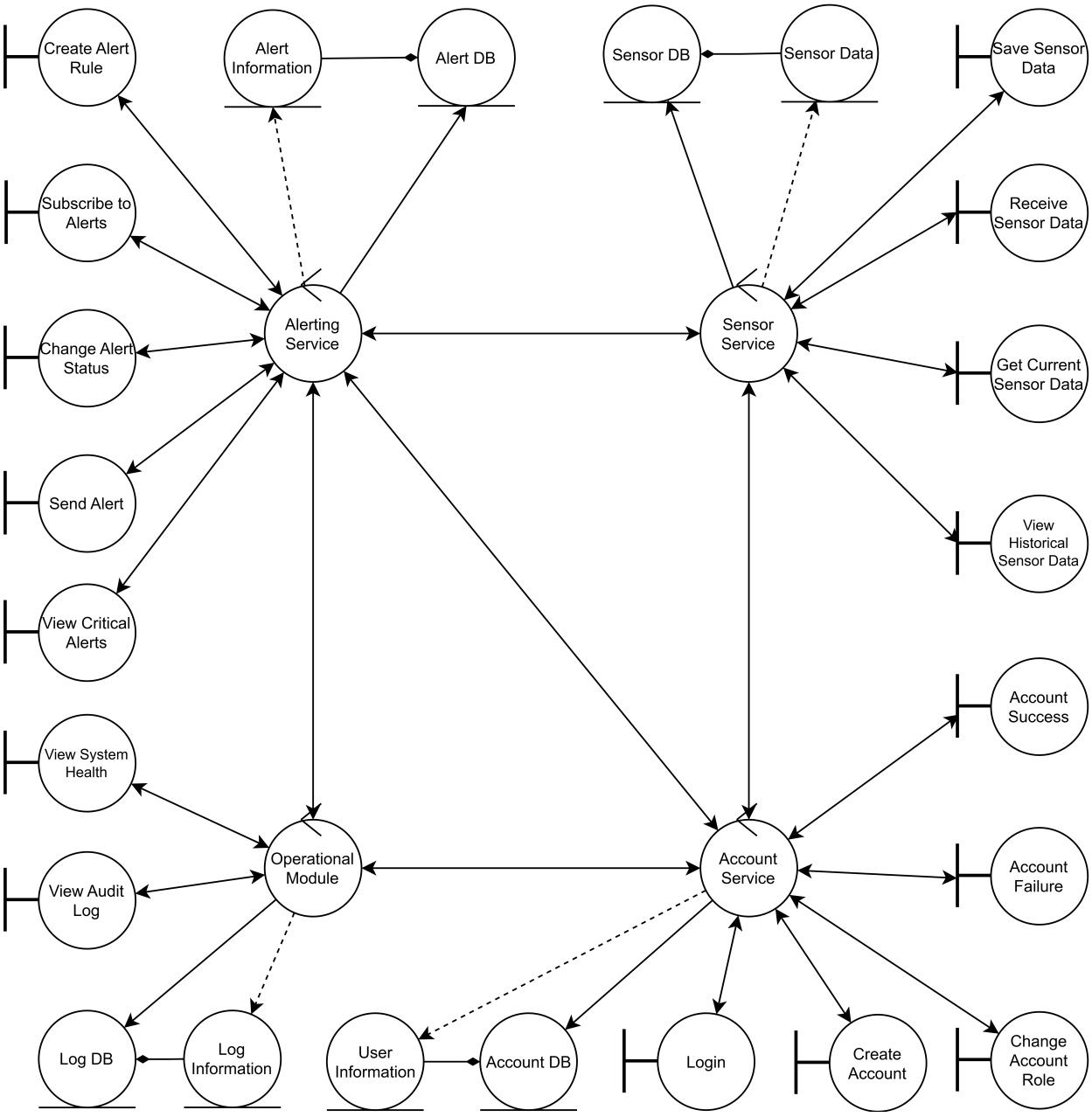

Figure 1: Analysis Class Diagram

3 Architectural Design

3.1 System Architecture

The SCEMAS system will utilize an interaction-oriented software architecture, specifically the Presentation-Abstract-Controller (PAC) architecture style. For this pattern, the presentation component represents the user interface and all user interaction, the abstraction represents the sensor data and all core functionality, and the controller is responsible for mediating communication between the presentation and abstraction components.

Within SCEMAS, there are several subsystems have been defined as the following:

Subsystem	**Purpose**	**Architectural Style**
Sensor Service	Ingest, validate, aggregate, and store relevant telemetry sensor data	Pipe and Filter
Alerting Service	Create alert rules, subscribe to alerts, send alerts, change alert status	Blackboard
Account Service	Creating account, logging-in, and handling Role-Based Access Control (RBAC)	Repository
Operational Module	Audit logs relevant to device lifecycle changes, and system health	Repository

The SCEMAS system will use PAC, Blackboard, and Repository architecture styles. The PAC architecture style was chosen for its compatibility with multiple input data streams. For SCEMAS, multiple environmental metrics will need to be tracked using various sensors. This concurrent input is directly applicable to the PAC architecture.

The sensor subsystem within SCEMAS will follow a pipe and filter architecture. A filter is justified for the sensor subsystem because it is able to read the environmental metrics data from its sensor input data stream. This will then be transformed and processed to ensure data is valid with plausible ranges and correct form. This transformed data stream will be passed through a pipe to the next filter where the data will be checked for critical values. Critical values will be flagged and the resulting data stream will be written to a data sink for storage. A pipe and filter architecture does not need to wait for the batched data as a whole to arrive and can start working as soon as the data arrives, this is important for SCEMAS as it revolves around the idea of working with real-time data. Thus, a pipe and filter architecture style is justified for this subsystem.

On the other hand, for the alerting service a blackboard style architecture was chosen as alerts will need to be sent out in response to changes in data (environmental metric levels). When the sensor data shows a critical level for an environmental metric, whether that be for temperature, air quality etc., the corresponding alert (client) is triggered and sent out. This scenario where there is a need for active data, and passive clients is consistent with the blackboard architecture style.

A repository architecture style was chosen for the account service subsystem. This is because, when interacting with the account service, users will have to go and actively interact with the data to get the corresponding response. Users will actively query the repository to receive information about their account, or another relevant data response.

Finally, a repository architecture style was also chosen for the operational module. This subsystem will have centralized data that contains information related to system health and history of audit logs. Clients will be able to access this passive data by generating reports or querying relevant statistics.

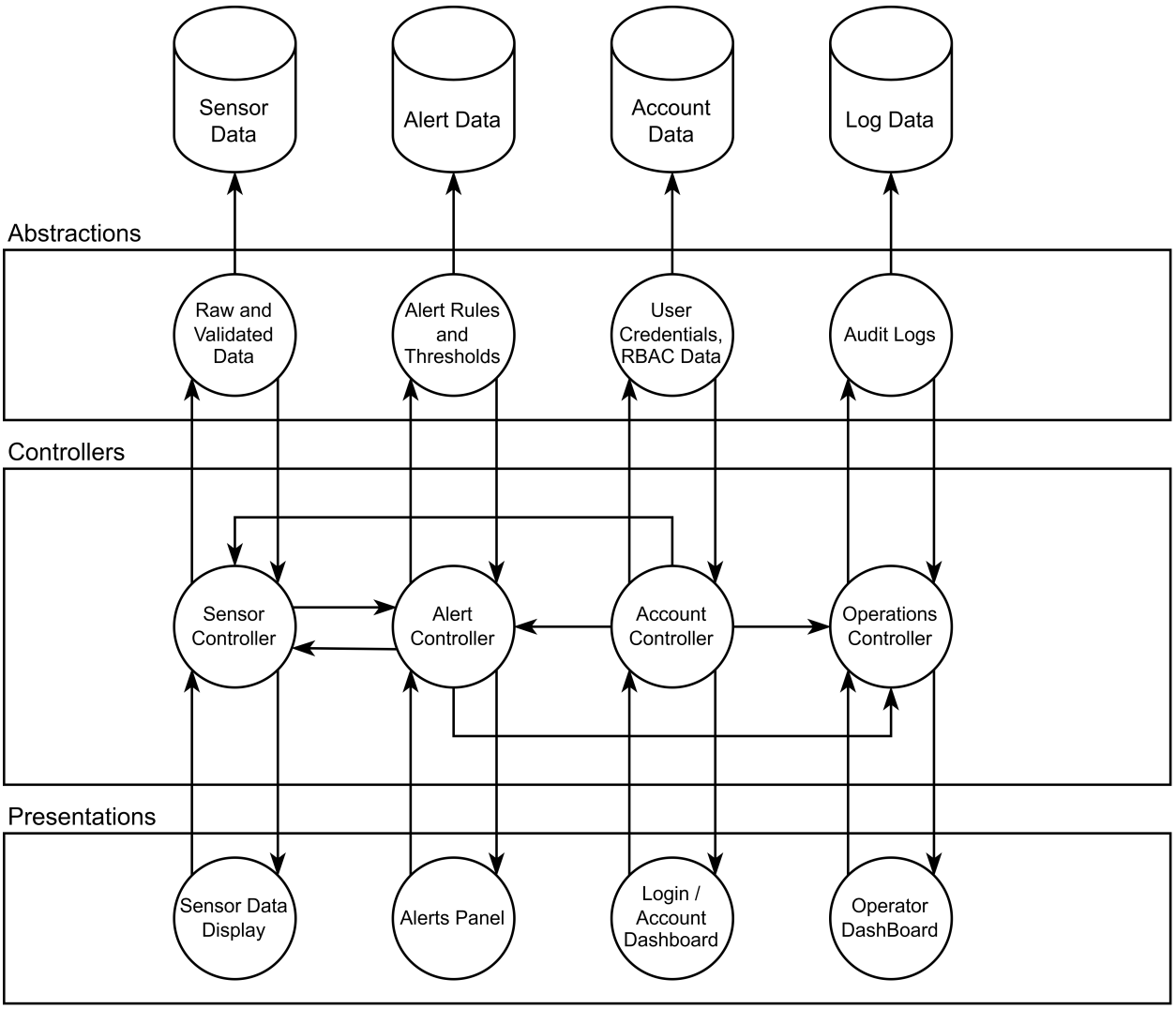

Figure 2: System Architecture

There were many other design alternatives that our team considered for the overall SCEMAS systems, such as a Repository architecture, a Blackboard architecture, and an MVC architecture.

Our team decided against a Repository architecture due to the requirements of SCEMAS. SCEMAS has components that need to react to data changes in real time, such as the alert trigger service. Since the Repository architecture cannot handle that reactive, event-driven behaviour, a Repository architecture should not be used in this scenario.

Our team also thought about using a Blackboard architecture - a style that is more "active" than a Repository architecture. However, we decided against this architecture because SCEMAS already has well-defined rules and alerting logic, which is not an ideal situation for a Blackboard architecture. A Blackboard architecture is best when there are multiple knowledge sources contributing partial solutions, which is not what's happening with SCEMAS.

Our team decided against MVC-I because in this system, it is crucial to separate concerns for different user roles. Since we have various users and RBAC, collapsing View and Controller together can get very messy

with the different views that are needed by different roles.

We thought about using an MVC-II architecture, however decided against it because of the shared model that it has to have. SCEMAS has four subsystems with different logic, data, and UI concerns, which can create tight coupling when combining them into one model. Using PAC keeps each subsystem and its data encapsulated in its own Abstraction, which results in lower coupling.

We eliminated the Process-Control Architecture Style as it is typically used for the control of embedded systems, which is not relevant to the system being created.

We also considered a Batch Sequential architectural style, however due to its reliance on batched data, it is not ideal to use this style when accessing real-time sensor data. In addition, the batch sequential style has no interactive interface and also has low throughput, which would not be beneficial to a system being accessed by multiple users.

We considered the Pipe and Filter Architecture Style, however, it does not allow for dynamic interactions. It would be beneficial for our system to work dynamically for users to create accounts and view alerts and sensor data.

3.2 Subsystems

The SCEMAS application is divided into several subsystems.

Sensor Service: This subsystem partakes in receiving, validating and storing telemetry sensor data. It processes data sent from the sensors to ensure it follows the correct format and is stored in a database. This subsystem interacts with the alerting service subsystem by providing environmental data that is used to produce alerts.

Alerting Service: This subsystem is responsible for creating and managing alerts. Data is constantly analyzed against predefined thresholds and when any threshold is triggered, the subsystem generates an alert and sends notifications to subscribed users. It also allows city operators to view, monitor and resolve any alerts. This subsystem communicates with both the public and operating module subsystems to send notifications and allow operators to carry administrative tasks respectively, as well as the sensor service for data input.

Account Service: This subsystem allows users to create, modify and delete their accounts and enforces user authorization. It implements Role Based Access Control (RBAC) ensuring that users have appropriate permissions depending on the user (e.g public user, city operator etc.). This subsystem provides secure access to the operational module.

Operating Module: This subsystem provides administrative functionalities for city operators. Through this, operators can monitor sensitive data, access audit logs and other regulatory tasks. It interacts with the account management subsystem to ensure only authorized users have access as well as alert subsystem to manage alerts.

4 Class Responsibility Collaboration (CRC) Cards

Class Name: Sensor Service (Controller)	
Responsibility:	**Collaborators:**
Knows Sensor Data Knows Sensor DB Knows Save Sensor Data Knows Receive Sensor Data Knows Get Current Sensor Data Knows View Historical Sensor Data Knows Alerting Service Knows Account Service	Sensor Data Sensor DB Save Sensor Data Receive Sensor Data Get Current Sensor Data View Historical Sensor Data Alerting Service Account Service

Class Name: Sensor Data (Entity)	
Responsibility:	**Collaborators:**
Knows Sensor DB Knows Sensor Service Knows Measurement Knows Unit Knows Location Knows Sensor ID	Sensor DB Sensor Service

Class Name: Sensor DB (Entity)	
Responsibility:	**Collaborators:**
Knows Sensor Data Knows Sensor Service	Sensor Data Sensor Service

Class Name: Save Sensor Data (Boundary)	
Responsibility:	**Collaborators:**
Knows Sensor Service Knows Sensor ID Handles saving sensor data to the database	Sensor Service

Class Name: Receive Sensor Data (Boundary)	
Responsibility:	**Collaborators:**
Knows Sensor Service Knows Sensor ID Handles receiving and validating sensor data from external sensors	Sensor Service

Class Name: Get Sensor Data (Boundary)	
Responsibility:	**Collaborators:**
Knows Sensor Service Knows Sensor ID Retrieves sensor data from the database	Sensor Service

Class Name: View Historical Sensor Data (Boundary)	
Responsibility:	**Collaborators:**
Knows Sensor Service Knows Sensor ID Handles click-event of "View Historical Data" button	Sensor Service

Class Name: Operational Module (Controller)	
Responsibility:	**Collaborators:**
Knows View System Health Knows View Audit Log Knows Log DB Knows Log Information Knows Alerting Service Knows Account Service	View System Health View Audit Log Log DB Log Information Alerting Service Account Service

Class Name: View System Health (Boundary)	
Responsibility:	**Collaborators:**
Knows Operational Module Handles click-event of "View System Health" button	Operational Module

Class Name: View Audit Log (Boundary)	
Responsibility:	**Collaborators:**
Knows Operational Module Handles click-event of "View Audit Log" button	Operational Module

Class Name: Log DB (Entity)	
Responsibility:	**Collaborators:**
Knows Log Information Knows Operational Module	Log Information Operational Module

Class Name: Log Information (Entity)	
Responsibility:	**Collaborators:**
Knows Log DB Knows Operational Module Knows Log ID Knows Author ID Knows Log Type Knows Time	Log DB Operational Module

Class Name: Account Service (Controller)	
Responsibility:	**Collaborators:**
Knows User Information Knows Account DB Knows Login Knows Create Account Knows Change Account Rule Knows Account Failure Knows Account Success Knows Operational Module Knows Alerting Service Knows Sensor Service	User Information Account DB Login Create Account Change Account Rule Account Failure Account Success Operational Module Alerting Service Sensor Service

Class Name: User Information (Entity)	
Responsibility:	**Collaborators:**
Knows Account DB Knows Account Service Knows User ID Knows Username Knows Password Knows Role	Account DB Account Service

Class Name: Account DB (Entity)	
Responsibility:	**Collaborators:**
Knows User Information Knows Account Service	User Information Account Service

Class Name: Login (Boundary)	
Responsibility:	**Collaborators:**
Knows Account Service Knows Username Knows Password Handles click-event of "login" button	Account Service

Class Name: Create Account (Boundary)	
Responsibility:	**Collaborators:**
Knows Account Service Knows Username Knows Password Handles click-event of "Create Account" button	Account Service

Class Name: Change Account Role (Boundary)	
Responsibility:	**Collaborators:**
Knows Account Service Knows Account ID Knows New Role Handles click-event of "Change Account Role" button	Account Service

Class Name: Account Failure (Boundary)	
Responsibility:	**Collaborators:**
Knows Account Service Handles "Time-out" event Handles "Saving Failure" event Handles "Creating account error" event	Account Service

Class Name: Account Success (Boundary)	
Responsibility:	**Collaborators:**
Knows Account Service Handles successful "Saving/Creating Account" events	Account Service

Class Name: Alerting Service (Controller)	
Responsibility:	**Collaborators:**
Knows Subscribe to Alerts Knows View Critical Alerts Knows Send Alert Knows Change Alert Status Knows Create Alert Rule Knows Alert DB Knows Alert Information Knows Account Service Knows Sensor Service Knows Operational Module	Subscribe to Alerts View Critical Alerts Send Alert Change Alert Status Create Alert Rule Alert DB Alert Information Account Service Sensor Service Operational Module

Class Name: Subscribe to Alerts (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Service Knows Alert Types Knows Locations Knows Account ID Handles click-event of "Subscribe to Alerts" button	Alerting Service

Class Name: View Critical Alerts (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Service Handles click-event of "View Critical Alerts"	Alerting Service

Class Name: Send Alert (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Service Knows Alert ID Handles "Send Alert" event	Alerting Service

Class Name: Change Alert Status (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Service Knows Alerting Status Knows Alert ID	Alerting Service

Class Name: Create Alert Rule (Boundary)	
Responsibility:	**Collaborators:**
Knows Alerting Service Knows Alert Type Knows Alert Threshold Handles click-event of "Create Alert Rule" button	Alerting Service

Class Name: Alert DB (Entity)	
Responsibility:	**Collaborators:**
Knows Alert Information Knows Alerting Service	Alert Information Alerting Service

Class Name: Alert Information (Entity)	
Responsibility:	**Collaborators:**
Knows Alert DB Knows Alerting Service Knows Alert ID Knows Type Knows Status Knows Severity Knows Start Date	Alert DB Alerting Service

A Division of Labour

Division of Labour sheet which indicates the contributions of each team member.

A.1 Bilal, Mohammad

- Section 2: Analysis Class Diagram
- Section 4: CRC Cards
- Porting document to LaTeX

A.2 Jessani, Kaiden

- Section 3.1: System Architecture

A.3 Ludwig, Christopher

- Section 1: Introduction
- Section 4: CRC Cards

A.4 Shahid, Irfan

- Section 3.2: Subsystems

A.5 Patel, Prutha

- Section 3.1: System Architecture